
Prompt engineering is dead.

Everyone agrees.

Almost no one has noticed that

"context engineering" is just four operations

– write, select, compress, isolate –

and that three of them belong to a router, not a human.

4

What we built to prove it.

WRITE

The human gate.

WRITE is what you still do: author the system message, the user query, the few-shot examples.

This is intent capture – you're encoding what the model should care about. It's not automatable because semantic intent requires a human who knows the domain.

This is where most teams stop. They write a prompt, call it engineering, ship it.

They're one operation in. There are three more.

STAYS HUMAN

SELECT

The router wakes up.

SELECT picks the minimal context that answers this query.

This is RAG, semantic search, graph traversal – any filter

that says 'include this chunk, exclude that one.'

Most teams stop here too. They call this 'done.'

The LLM still sees bloat. Latency still scales with corpus

size, not query specificity.

SELECT is necessary. It is not sufficient.

COMPRESS

Where token spend collapses.

COMPRESS removes redundancy within the selected chunks.

Strip boilerplate. Summarize repeated patterns. Extract only the signal – 'line 247 is the function you need,' not the whole 500-line file.

Compression ratio on typical doc/code sets: 3:1 to 8:1. This is where ContextWeave enters.

Most teams skip this. 'LLMs are cheap now.' They're not.

ISOLATE

The guard rail. Almost never built.

ISOLATE injects metadata barriers so the model can't conflate contexts from different sources.

Example: 'best practices from doc A' and 'your codebase from doc B' need explicit namespace tags. Without them, the model hallucinates 'best practice X applies to your code' – when it doesn't.

ISOLATE = semantic namespace enforcement.

Costs nearly nothing. Catches 60%+ of confabulation errors in multi-context settings. Underrated.

The automation boundary.

Which operations belong to a router. Which stay human.

STAYS HUMAN	AUTOMATES
WRITE Intent is human-defined. Can't commoditize context.	SELECT Semantic relevance is measurable. COMPRESS Redundancy detection is algorithmic. ISOLATE Tagging schema defined once, applied at scale.

ContextWeave (Apache 2.0, aiweave.org) – implements all three.

The framework

Context engineering = a system you can meter.

WRITE

SELECT

COMPRESS

ISOLATE

Prompt engineering was always a proxy for 'get the context right.'

Now the three mechanical operations can be automated.

SELECT, COMPRESS, ISOLATE belong to the router.

WRITE – the only one that requires your intent – stays yours.

91% token reduction on production agent loops.

(ContextWeave / ExpertiseRAG – aiweave.org, Apache 2.0)

If your team runs multi-agent loops or RAG: what's your token spend per invocation – and are you tracking compression ratios at all?

Save this. The four operations are free to codify in your own stack.